Compte Rendu en techniques d’indexation et de Recherche Multimédia

Hamdi Abderrahim
26/02/2026

TP1 : Indexation et Recherche d'Images Par le Contenu(CBIR)

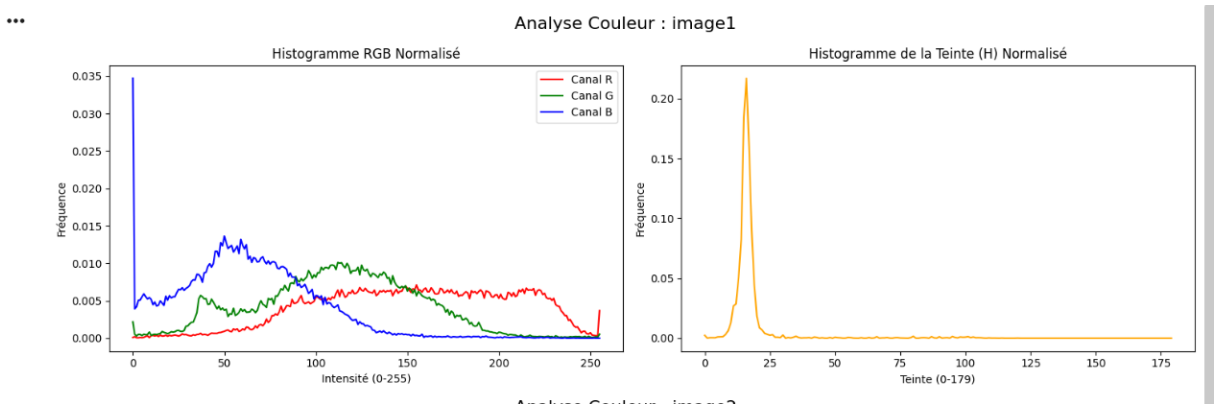
Partie 2 : Extraction des descriptions visuels:

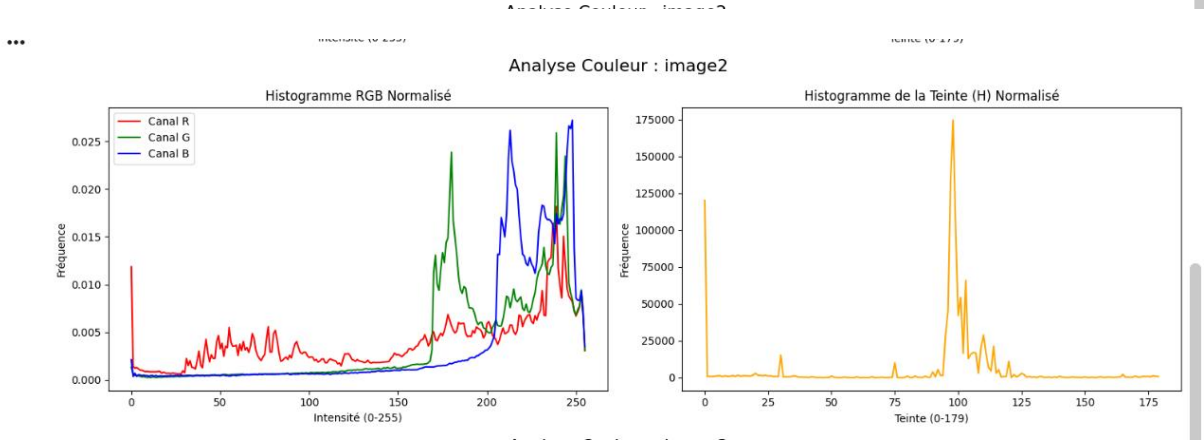
2.1 Description de couleur :

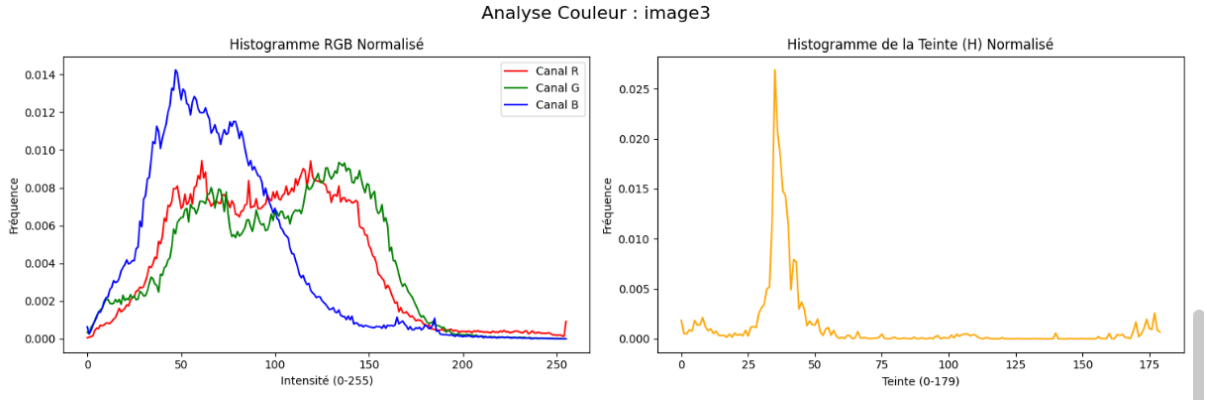
```python
import cv2
import matplotlib.pyplot as plt
import numpy as np
images = ["/content/a.jpg", "/content/b.jpg", "/content/c.jpg", "/content/d.jpg", "/content/e.jpg"]
images_ind = ["image1", "image2", "image3", "image4", "image5"]
for idx, path in enumerate(images):
    img_bgr = cv2.imread(path)
    if img_bgr is None:
        print(f"Erreur de lecture pour {path}")
        continue
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    fig, axs = plt.subplots(1, 2, figsize=(15, 5))
    fig.suptitle(f"Analyse Couleur : {images_ind[idx]}", fontsize=16)
    colors = ('r', 'g', 'b')
    for i, col in enumerate(colors):
        hist = cv2.calcHist([img_rgb], [i], None, [256], [0, 256])
        hist_norm = hist / hist.sum()
        axs[0].plot(hist_norm, color=col, label=f'Canal {col.upper()}')
    axs[0].set_title("Histogramme RGB Normalisé")
    axs[0].set_xlabel("Intensité (0-255)")
    axs[0].set_ylabel("Fréquence")
    axs[0].legend()
```

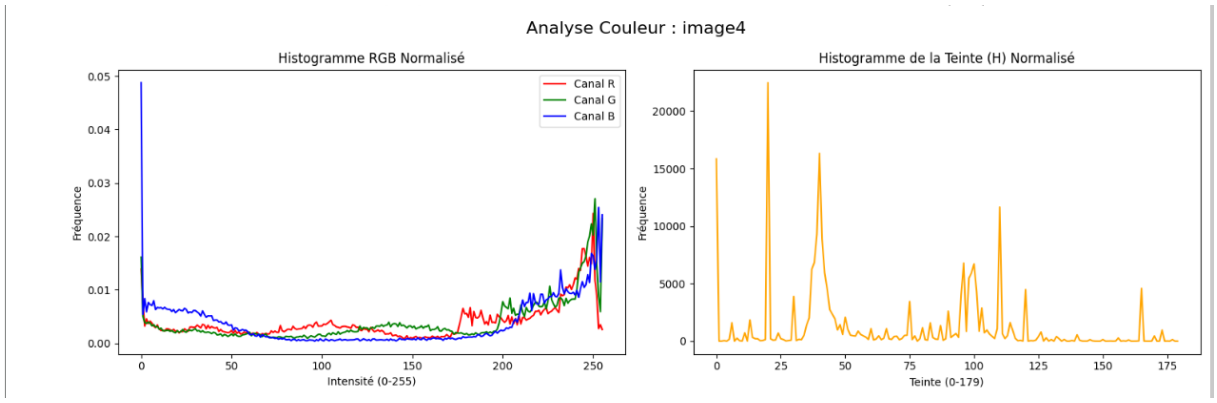
```python
    axs[0].legend()
    hist_h = cv2.calcHist([img_hsv], [0], None, [180], [0, 180])
    hist_h_norm = hist_h / hist_h_norm.sum() if 'hist_h_norm' in locals() else hist_h / hist_h.sum()
    axs[1].plot(hist_h_norm, color='orange')
    axs[1].set_title("Histogramme de la Teinte (H) Normalisé")
    axs[1].set_xlabel("Teinte (0-179)")
    axs[1].set_ylabel("Fréquence")
    plt.tight_layout()
    plt.show()
```

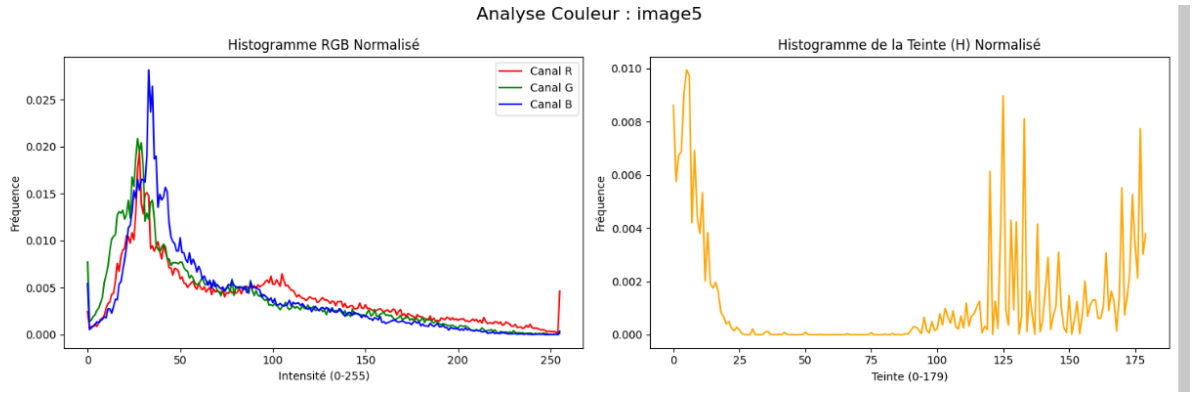
Résultat :


Analyse Couleur : image1
Histogramme RGB Normalisé
Canal R
Canal G
Canal B
Fréquence
Intensité (0-255)
Histogramme de la Teinte (H) Normalisé
Fréquence
Teinte (0-179)


Analyse Couleur : image2
Histogramme RGB Normalisé
Canal R
Canal G
Canal B
Fréquence
Intensité (0-255)
Histogramme de la Teinte (H) Normalisé
Fréquence
Teinte (0-179)


Analyse Couleur : image3
Histogramme RGB Normalisé
Canal R
Canal G
Canal B
Fréquence
Intensité (0-255)
Histogramme de la Teinte (H) Normalisé
Fréquence
Teinte (0-179)


Analyse Couleur : image4
Histogramme RGB Normalisé
Canal R
Canal G
Canal B
Fréquence
Intensité (0-255)
Histogramme de la Teinte (H) Normalisé
Fréquence
Teinte (0-179)

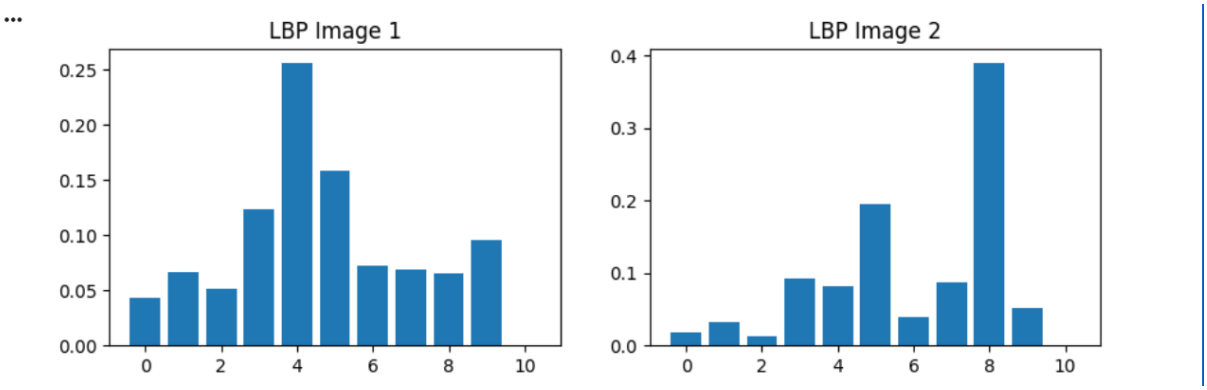


2.2 Description de texture :

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
from skimage.feature import local_binary_pattern
P, R = 8, 1
def extract_texture_lbp(image_path):
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    lbp = local_binary_pattern(img, P, R, method="uniform")
    (hist, _) = np.histogram(lbp.ravel(), bins=np.arange(0, 12), range=(0, 11))
    hist = hist.astype("float")
    hist /= (hist.sum() + 1e-7)
    return hist
lbp_1 = extract_texture_lbp(images[0])
lbp_2 = extract_texture_lbp(images[1])

plt.figure(figsize=(10, 3))
plt.subplot(1, 2, 1); plt.bar(range(len(lbp_1)), lbp_1); plt.title("LBP Image 1")
plt.subplot(1, 2, 2); plt.bar(range(len(lbp_2)), lbp_2); plt.title("LBP Image 2")
plt.show()
```

Résultat :



Partie 3 : Indexation de la base d'images :

```
import pandas as pd
indexing_table = []
for idx, path in enumerate(images):
    img_bgr = cv2.imread(path)
    img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    hist_h = cv2.calcHist([img_hsv], [0], None, [180], [0, 180])
    hist_h /= (hist_h.sum() + 1e-7)
    lbp_feat = extract_texture_lbp(path)
    features = np.concatenate([hist_h.flatten(), lbp_feat])
    indexing_table.append({
        'id': images_ind[idx],
        'path': path,
        'features': features
    })
features_array = np.array([item['features'] for item in indexing_table])
np.save('indexation_base.npy', features_array)
print("Table d'indexation construite :", len(indexing_table), "images indexées.")
```

```
Table d'indexation construite : 5 images indexées.
```

Fichier construit :

indexation_base.npy

```
import numpy as np
data = np.load("/content/indexation_base.npy")
print(data)
```

```
[[2.36666668e-03 9.99999975e-05 4.33333335e-04 3.99999990e-04
  3.99999990e-04 1.08333328e-03 1.15000003e-03 1.26666669e-03
  3.16666672e-03 6.23333314e-03 1.25500001e-02 2.68833339e-02
  2.85999998e-02 5.05666658e-02 8.22833329e-02 1.84333339e-01
  2.16900006e-01 1.60666659e-01 9.06166658e-02 4.51666676e-02
  1.83499996e-02 8.71666707e-03 6.85000001e-03 4.05000011e-03
  3.25000007e-03 2.45000003e-03 2.93333340e-03 8.16666696e-04
  8.16666696e-04 5.16666682e-04 2.63333344e-03 0.00000000e+00
  5.83333313e-04 2.33333340e-04 7.99999980e-04 1.61666668e-03
  6.16666686e-04 1.66666665e-04 3.33333330e-04 3.33333330e-04
  5.16666682e-04 4.99999987e-05 9.50000016e-04 6.50000002e-04
  3.33333337e-05 3.83333332e-04 4.99999987e-05 0.00000000e+00
  5.83333313e-04 0.00000000e+00 7.83333322e-04 4.83333337e-04
```

Partie 4 : Recherche par similarité:

```python
from scipy.spatial import distance
def chi2_distance(A, B):
    return 0.5 * np.sum([((a - b) ** 2) / (a + b + 1e-10) for (a, b) in zip(A, B)])
query_idx = 0
query_vec = indexing_table[query_idx]['features']

results = []
for item in indexing_table:
    dist = distance.euclidean(query_vec, item['features'])
    results.append((item['id'], dist, item['path']))
results.sort(key=lambda x: x[1])
print(f"Top images pour la requête : {images_ind[query_idx]}")
plt.figure(figsize=(15, 5))
for i, res in enumerate(results[:5]):
    plt.subplot(1, 5, i+1)
    img_res = cv2.imread(res[2])
    plt.imshow(cv2.cvtColor(img_res, cv2.COLOR_BGR2RGB))
    plt.title(f"{res[0]}\nDist: {res[1]:.2f}")
    plt.axis('off')
plt.show()
```

Résultat :

```
Top images pour la requête : image1
```

Partie 5 : Evaluation du système:

5.1 Calcul des métriques :

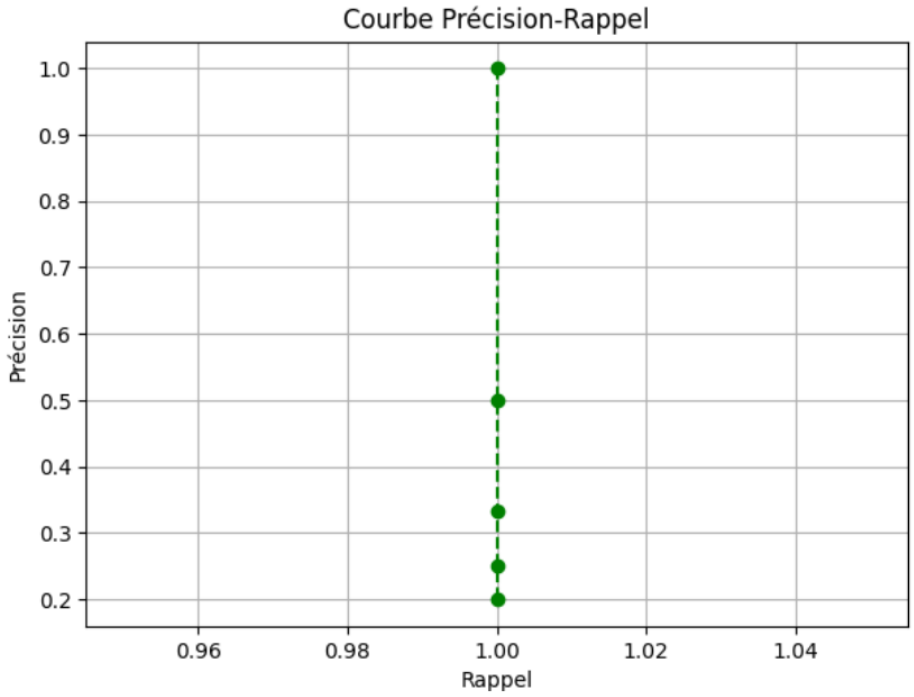
```python
def get_label(img_name):
    return img_name.split('image')[1][0]
query_label = get_label(images_ind[query_idx])
total_relevant = sum(1 for name in images_ind if get_label(name) == query_label)
precisions = []
recalls = []
relevant_found = 0
for i, res in enumerate(results):
    if get_label(res[0]) == query_label:
        relevant_found += 1
    precisions.append(relevant_found / (i + 1))
    recalls.append(relevant_found / total_relevant)
```

5.2 Courbes précision-rappel :

```
# 5.2 Traçage courbes P/R
plt.figure(figsize=(7, 5))
plt.plot(recalls, precisions, marker='o', color='green', linestyle='--')
plt.title("Courbe Précision-Rappel")
plt.xlabel("Rappel")
plt.ylabel("Précision")
plt.grid(True)
plt.show()
```

Résultat :

...



Partie 6 : Analyse et discussion:

--- RÉPONSES AUX QUESTIONS ---

1/

Le descripteur combiné (HSV + LBP) est généralement le plus robuste car il allie couleur et structure

2/

La distance Euclidienne est rapide, mais la distance Chi-square est souvent plus précise pour comparer des histogrammes

3/

Le 'fossé sémantique' (Semantic Gap). Le système peut trouver des images visuellement proches (mêmes couleurs) mais n'ayant aucun rapport de sens